Assignment – 8

Submitted by ~ Ishika Dutta (CT_CSI_DV_5572)

Task 8.1

Configure dashboard and queries for work items.

- *Navigate to Azure DevOps Project > Boards > Queries*
    - *Create custom queries (e.g., Active Bugs, Assigned to me, Sprint Backlog).*
- *Go to Dashboards > New Dashboard*
    - *Add widgets: Query Results, Sprint Burndown, Team Velocity, Pie Chart, etc.*
    - *Link your saved queries to widgets.*
    - *Share dashboard with your team.*

Task 8.2

Use pipeline variables while configuring pipelines.

azure-pipelines.yml or classic UI:

```
variables:
  imageName: 'myapp'
  tag: '$(Build.BuildId)'
```

Use in tasks:

```
- script: echo $(imageName):$(tag)
```

Task 8.3

Use variable and task groups in pipelines and set scopes for different stages.

Scope Variables to Stages

```
variables:
- group: 'MyVarGroup'

stages:
- stage: Build
  variables:
    environment: 'dev'
```

Task 8.4

Create a service connection.

Go to Project Settings > Service Connections

- *Choose Azure Resource Manager → Service principal (automatic) → Select subscription.*
- *Provide name like AzureConnection.*
- *Authorize with your Azure credentials.*

Task 8.5

Create a Linux/Windows self-hosted agent.

On the target machine:

- *Go to Project Settings > Agent Pools*
- *Click New agent, choose OS → Download agent package.*
- *Run the setup:*

```
./config.sh --unattended --url https://dev.azure.com/YourOrg --auth pat --token <PAT> --pool Default --agent my-linux-agent
./svc.sh install
./svc.sh start

```

Task 8.6

Apply pre and post-deployment approvers in the release pipeline.

In Pipelines > Releases:

- *Click on your Stage → Enable Pre-deployment conditions → Add approvers.*
- *Similarly, set Post-deployment conditions → Add reviewers for validation.*

Task 8.7

Create a CI/CD pipeline to build and push a Docker image to ACR and deploy to AKS.

```yaml
trigger:
  - main

variables:
    azureSubscription: 'AzureConnection'
    imageName: 'myapp'
    containerRegistry: 'myacr.azurecr.io'
    kubernetesCluster: 'myAKSCluster'
    namespace: 'default'

stages:
- stage: Build
  jobs:
  - job: Build
    steps:
    - task: Docker@2
      inputs:
        containerRegistry: '$(azureSubscription)'
        repository: '$(imageName)'
        command: 'buildAndPush'
        tags: '$(Build.BuildId)'

  - stage: Deploy
  jobs:
  - deployment: DeployToAKS
    environment: aks
    strategy:
      runOnce:
        deploy:
          steps:
          - task: Kubernetes@1
            inputs:
              connectionType: 'Azure Resource Manager'
              azureSubscription: '$(azureSubscription)'
              azureResourceGroup: 'my-rg'
              kubernetesCluster: '$(kubernetesCluster)'
              namespace: '$(namespace)'
              command: apply
              useConfigurationFile: true
              configuration: 'manifests/deployment.yaml'

```

Task 8.8

Create a CI/CD pipeline to build and push a Docker image to ACR and deploy to ACI.

```yaml
trigger:
  - main

variables:
    azureSubscription: 'AzureConnection'
    resourceGroup: 'aci-rg'
    containerName: 'aci-container'
    containerRegistry: 'myacr.azurecr.io'
    imageName: 'myapp'
    location: 'eastus'

stages:
- stage: Build
  jobs:
  - job: BuildPush
    steps:
    - task: Docker@2
      inputs:
        containerRegistry: '$(azureSubscription)'
        repository: '$(imageName)'
        command: 'buildAndPush'
        tags: '$(Build.BuildId)'

  - stage: Deploy
  jobs:
  - deployment: DeployACI
    environment: aci
    strategy:
      runOnce:
        deploy:
          steps:
          - task: AzureCLI@2
            inputs:
              azureSubscription: '$(azureSubscription)'
              scriptType: 'bash'
              scriptLocation: 'inlineScript'
              inlineScript: |
                az container create \
                  --resource-group $(resourceGroup) \
                  --name $(containerName) \
                  --image $(containerRegistry)/$(imageName):$(Build.BuildId) \
                  --cpu 1 --memory 1 \
                  --dns-name-label $(containerName)-dns \
                  --ports 80

```

Task 8.9

Create a CI/CD pipeline to build a .NET application and deploy it to Azure App Service.

```yaml
trigger:
  - main

variables:
    azureSubscription: 'AzureConnection'
    appName: 'dotnet-app-service'
    webAppPackage: '$(System.DefaultWorkingDirectory)/**/*.zip'

stages:
- stage: Build
  jobs:
  - job: Build
    steps:
    - task: UseDotNet@2
      inputs:
        packageType: 'sdk'
        version: '6.x.x'

    - script: |
        dotnet publish -c Release -o $(Build.ArtifactStagingDirectory)
      displayName: 'Publish .NET App'

    - task: ArchiveFiles@2
      inputs:
        rootFolderOrFile: '$(Build.ArtifactStagingDirectory)'
        includeRootFolder: false
        archiveType: zip
        archiveFile: '$(Build.ArtifactStagingDirectory)/app.zip'

    - task: PublishBuildArtifacts@1
      inputs:
        pathToPublish: '$(Build.ArtifactStagingDirectory)/app.zip'
        artifactName: 'drop'

- stage: Deploy
  jobs:
  - deployment: DeployAppService
    environment: 'production'
    strategy:
      runOnce:
        deploy:
          steps:
          - task: AzureWebApp@1
            inputs:
              azureSubscription: '$(azureSubscription)'
              appName: '$(appName)'
              package: '$(Pipeline.Workspace)/drop/app.zip'
```

Task 8.10

Create a CI/CD pipeline to build a React application and deploy it to an Azure virtual machine.

```yaml
trigger:
  - main

variables:
    vmIp: 'xx.xx.xx.xx'
    sshUsername: 'azureuser'
    sshPassword: '$(sshPassword)'  # stored securely
    appDirectory: '/var/www/reactapp'

stages:
- stage: Build
  jobs:
  - job: BuildReact
    steps:
    - task: NodeTool@0
      inputs:
        versionSpec: '16.x'

    - script: |
        npm install
        npm run build
      displayName: 'Build React App'

    - task: CopyFiles@2
      inputs:
        SourceFolder: 'build'
        Contents: '**'
        TargetFolder: '$(Build.ArtifactStagingDirectory)/build'

    - task: PublishBuildArtifacts@1
      inputs:
        pathToPublish: '$(Build.ArtifactStagingDirectory)/build'
        artifactName: 'reactapp'

- stage: Deploy
  jobs:
  - job: DeployToVM
    steps:
    - task: DownloadBuildArtifacts@0
      inputs:
        artifactName: 'reactapp'

    - task: CopyFilesOverSSH@0
      inputs:
        sshEndpoint: 'MyLinuxVMServiceConnection'
        sourceFolder: '$(Pipeline.Workspace)/reactapp'
        targetFolder: '$(appDirectory)'
```